

Phase 1 - Week 2 - Lesson Day 2

Data Selection & Filtering

Today's Objectives

Yesterday you loaded a dataset and got a first look at it. Today is about actually pulling out the specific rows and columns you need - the Pandas equivalent of filtering a spreadsheet or writing a SQL WHERE clause. This is one of the most-used skills in the entire course.

- Select individual columns and multiple columns
- Understand the difference between `.loc` and `.iloc`
- Filter rows using single and combined conditions
- Sort data by one or more columns

1. Selecting Columns

Definition: You can pull out one or more columns from a DataFrame using square brackets. Selecting a single column with one set of brackets returns a **Series**. Selecting multiple columns with a list inside the brackets (double brackets) returns a full **DataFrame**.

```
df["age"]          # select one column -> returns a Series
df[["name", "age"]] # select multiple columns -> returns a DataFrame (note the double brackets)
```

The double brackets are a common early source of confusion: the outer bracket is the normal selection syntax, and the inner bracket is a Python list of the column names you want.

2. `.loc` and `.iloc`

Definition: `.loc` selects rows and columns by *label* (the actual index value or column name). `.iloc` selects rows and columns by *integer position* (0, 1, 2, ...), regardless of what the labels actually are - similar to how you index a NumPy array or Python list.

```
df.loc[0]          # the row labeled 0
df.loc[0, "name"]   # value at row label 0, column 'name'
df.loc[0:2, "name"] # rows labeled 0 through 2 (INCLUSIVE), column 'name'

df.iloc[0]          # the first row, by position
df.iloc[0, 1]       # first row, second column, by position
df.iloc[0:3, 0:2]   # first 3 rows, first 2 columns, by position (EXCLUSIVE of end)
```

The key difference to remember: `.loc` slicing is *inclusive* of the end label (0:2 gives you rows 0, 1, AND 2). `.iloc` slicing behaves like normal Python slicing - *exclusive* of the end position (0:3 gives you positions 0, 1, and 2, but not 3). This difference trips up almost everyone at first, so it's worth re-reading until it clicks.

3. Conditional Filtering

Definition: Just like boolean masking with NumPy arrays, you can filter a DataFrame's rows using a condition written inside square brackets. This is the direct equivalent of a SQL **WHERE** clause or applying a filter to a column in Excel/Power BI.

```
adults = df[df["age"] >= 18]           # keep only rows where age is 18 or older
somalis = df[df["city"] == "Mogadishu"] # keep only rows where city equals 'Mogadishu'
```

You can combine multiple conditions using **&** (and) or **|** (or) - just like NumPy boolean masking, each condition must be wrapped in its own parentheses.

```
young_adults = df[(df["age"] >= 18) & (df["age"] <= 30)]
# keeps rows where age is BOTH 18 or older AND 30 or younger
```

A common mistake is using Python's normal **and/or** keywords instead of **&|** - this does not work correctly with Pandas conditions and will raise an error. Always use **&** and **|**, with parentheses around each condition.

4. Sorting

Definition: **.sort_values()** reorders the rows of a DataFrame based on the values in one or more columns - the same idea as sorting a column in Excel or using ORDER BY in SQL.

```
df.sort_values("age")           # sort ascending by age (smallest to largest)
df.sort_values("age", ascending=False) # sort descending (largest to smallest)
df.sort_values(["city", "age"]) # sort by city first, then by age within each city
```

Today's Exercise

Continue using the same dataset you loaded in Day 1. Open **day-2-selection-filtering.ipynb** and complete the questions below.

Exercise Questions (9 total)

1. Select one column

Select a single numeric column from your dataset and print its first 10 values.

2. Select multiple columns

Select 3 columns of your choice together and print the first 5 rows of just those columns.

3. Row by label with .loc

Use **.loc** to print the row at index label 5 (or the closest valid label in your dataset).

4. Row range with .loc

Use **.loc** to print rows 0 through 4 (inclusive) for two columns of your choice.

5. Row range with .iloc

Use .iloc to print the first 5 rows and first 3 columns by position, and compare the result to Question 4.

6. Single condition filter

Filter your dataset to keep only rows matching one condition of your choice (e.g. a numeric column above a certain value, or a category matching a specific value).

7. Combined condition filter

Filter your dataset using two combined conditions with & (for example, a numeric column between two values).

8. Sort ascending and descending

Sort your dataset by one numeric column in ascending order, then again in descending order, and print the top 5 rows of each.

9. Multi-column sort

Sort your dataset by two columns at once (one categorical, one numeric) and print the first 10 rows of the result.

How to Approach It

Before writing each filter, say the condition out loud in plain English first ("rows where age is over 30 and city is Mogadishu"), then translate it directly into the & / | syntax. This habit prevents most of the common mistakes with combined conditions.

Once you and Halimo have completed the exercises, bring your answers back to the lesson chat and we'll review them before moving to Lesson Day 3 (Data Cleaning).